

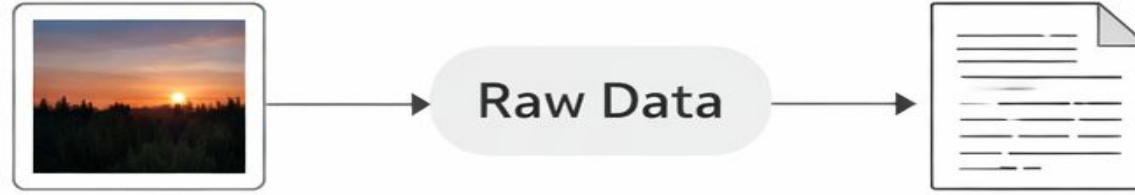
Chapter 4

REPRESENTATION LEARNING OVERVIEW

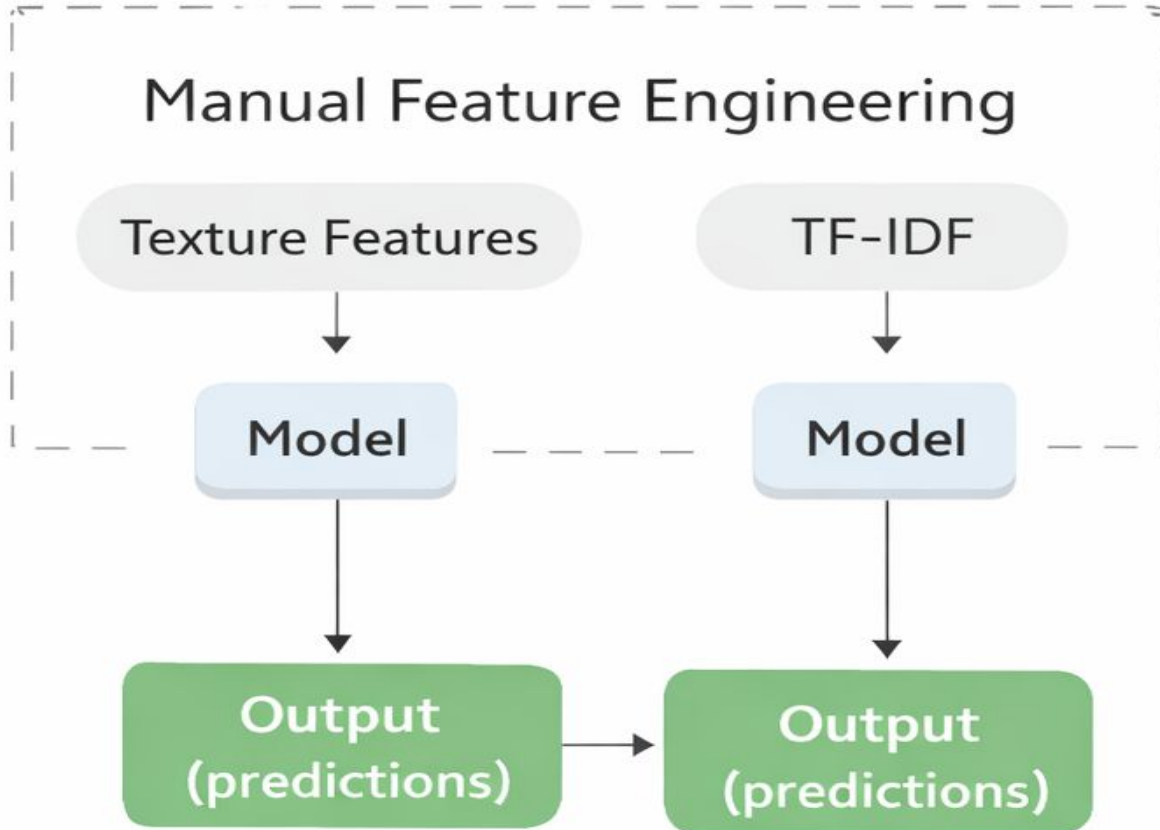
Representation Learning – Overview

- ❑ **Representation Learning** is a branch of machine learning that focuses on automatically discovering meaningful features (representations) from raw data. Instead of manually designing features, the model learns useful patterns and structures directly from the input data.
- ❑ In traditional machine learning, feature engineering is done manually (e.g., texture features in images, TF-IDF in text). In representation learning, the system learns these features automatically through layered transformations.

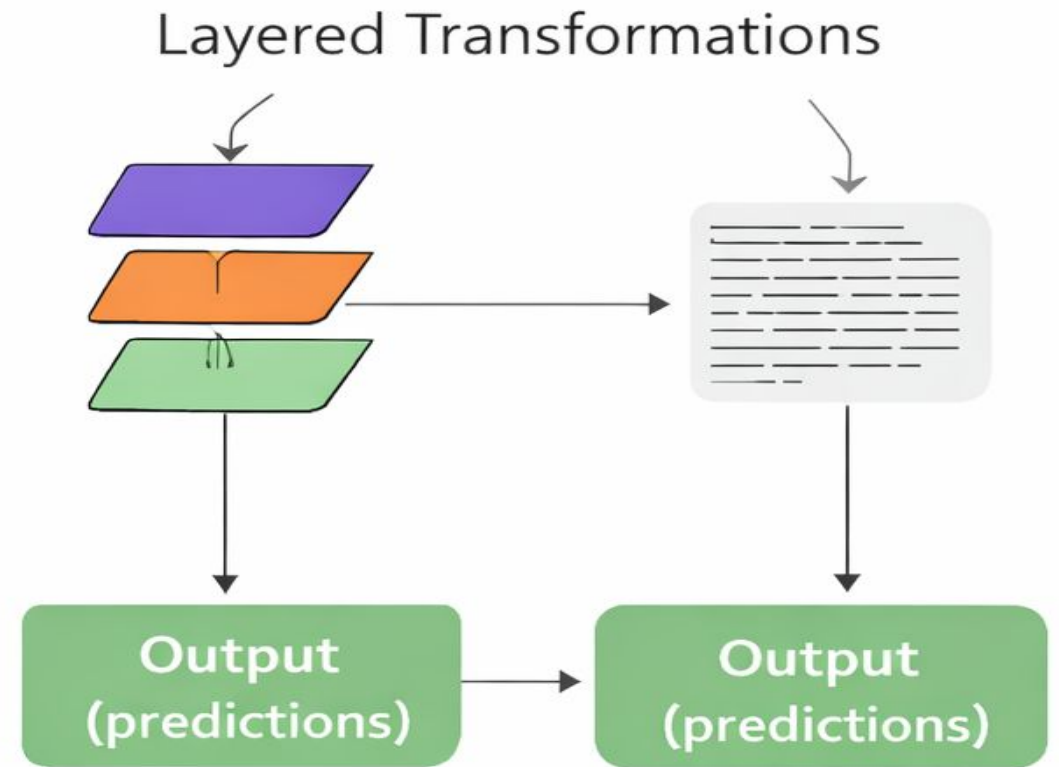
Representation Learning



Traditional Machine Learning



Representation Learning



Why Representation Learning is Important?

- Reduces manual feature engineering
- Learns hierarchical features (low-level → high-level)
- Improves performance in complex data (images, speech, text)
- Enables deep learning architectures

How It Works

Representation learning transforms raw input data into a more abstract and informative form:

Raw Data → Feature Extraction Layers → Learned Representation → Final Task
(Classification/Regression)

For example:

- Image pixels → edges → shapes → objects
- Text → word embeddings → semantic meaning
- Audio signals → frequency patterns → speech recognition

Types of Representation Learning

1. Supervised Representation Learning

Learns features using labeled data (e.g., CNN for image classification).

2. Unsupervised Representation Learning

Learns features without labels (e.g., Autoencoders, PCA).

3. Self-Supervised Learning

Learns representations using automatically generated labels (e.g., contrastive learning).

4. Deep Representation Learning

Uses deep neural networks to learn hierarchical features.

Examples

- **Word2Vec / GloVe** → Word embeddings
- **Autoencoders** → Feature compression
- **CNNs** → Image feature extraction
- **Transformers** → Context-aware language representation

Difference from Reinforcement Learning

Representation Learning	Reinforcement Learning
Focuses on feature extraction	Focuses on decision making
Learns data representations	Learns optimal actions
Used in classification, NLP, vision	Used in control, robotics, games

Applications

- Computer Vision
- Natural Language Processing
- Speech Recognition
- Medical Imaging
- Recommendation Systems

Machine Learning on Graphs

Machine Learning on Graphs : Background and Traditional Approaches

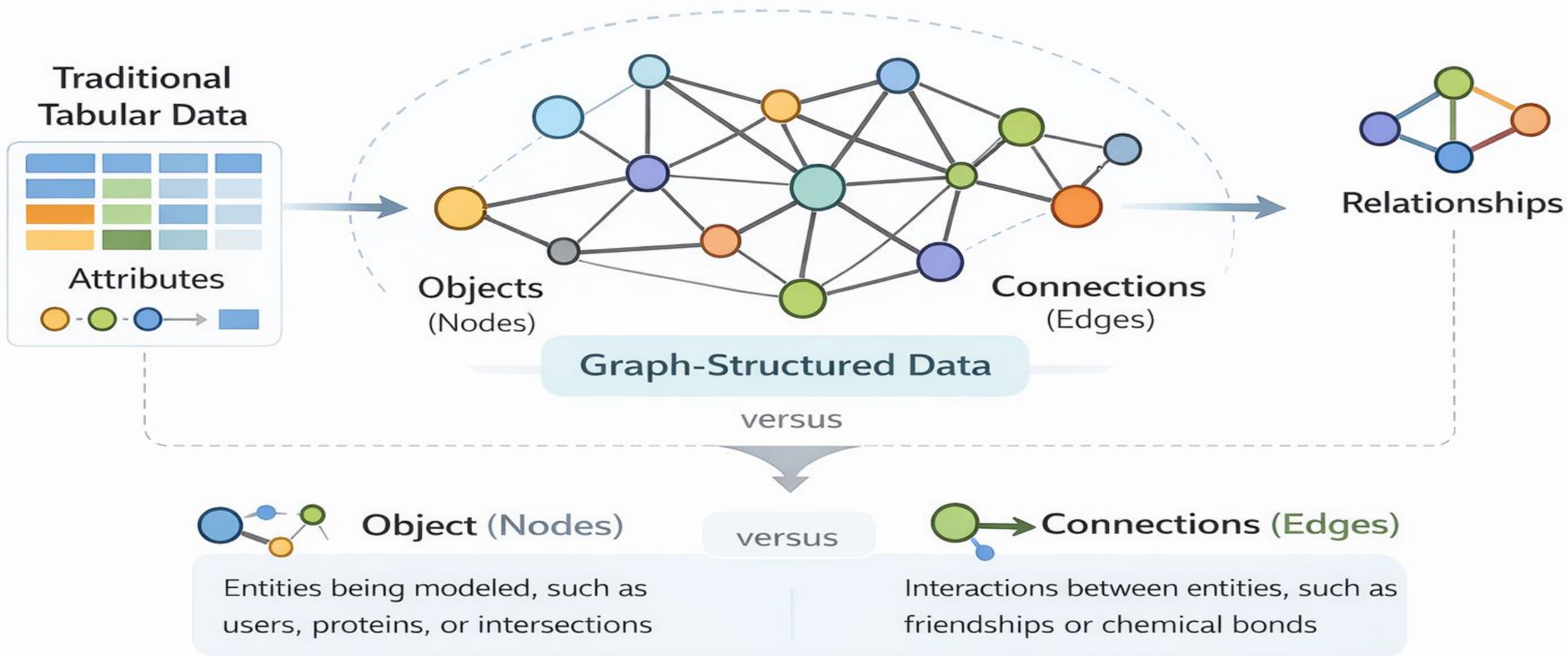
Machine Learning on Graphs focuses on analyzing and learning from graph-structured data, where relationships between entities are as important as the entities themselves. Unlike traditional tabular data, graphs explicitly model connections (edges) between objects (nodes).

Graphs are widely used in:

- ❑ Social networks (users connected by friendships)**
- ❑ Biological networks (genes, proteins interactions)**
- ❑ Traffic networks (intersections and roads)**
- ❑ Financial fraud detection (transaction networks)**
- ❑ Knowledge graphs (entity relationships)**

Machine Learning on Graphs

Focuses on analyzing and learning from graph-structured data, where relationships between entities are as important as **the entities themselves**.



Background of Graph-Based Learning

A graph is mathematically defined as:

$$G = (V, E)$$

Where:

- $V \rightarrow$ Set of nodes (vertices)
- $E \rightarrow$ Set of edges (connections)

Nodes may have features:

$$X \in \mathbb{R}^{|V| \times d}$$

Edges may also have weights or attributes.

Why Traditional ML Fails Directly?

Traditional ML assumes:

- ☐ Independent samples
- ☐ Fixed-length feature vectors
- ☐ No relational dependency

But in graphs:

- ☐ Data points are interdependent
- ☐ Structure matters
- ☐ Connectivity influences prediction

Example:

In fraud detection, one suspicious account connected to many fraudsters increases its fraud probability.

Functionality of Machine Learning on Graphs

Core Functional Components

The functionality can be understood in five major stages

① Graph Construction

First, data is represented as a graph:

$$G = (V, E, X)$$

Where:

- $V \rightarrow$ Nodes (entities)
- $E \rightarrow$ Edges (relationships)
- $X \rightarrow$ Node feature matrix

Example:

- Social network $\rightarrow$ Users (nodes), Friendships (edges)
- Traffic network $\rightarrow$ Intersections (nodes), Roads (edges)

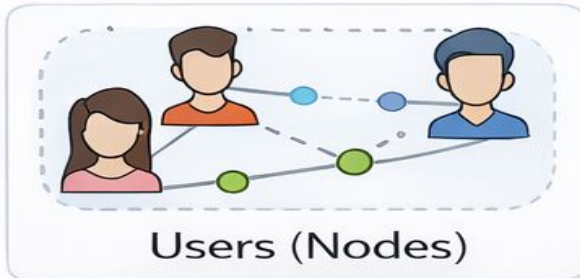
① Graph Construction

First, data is represented as a **graph**:

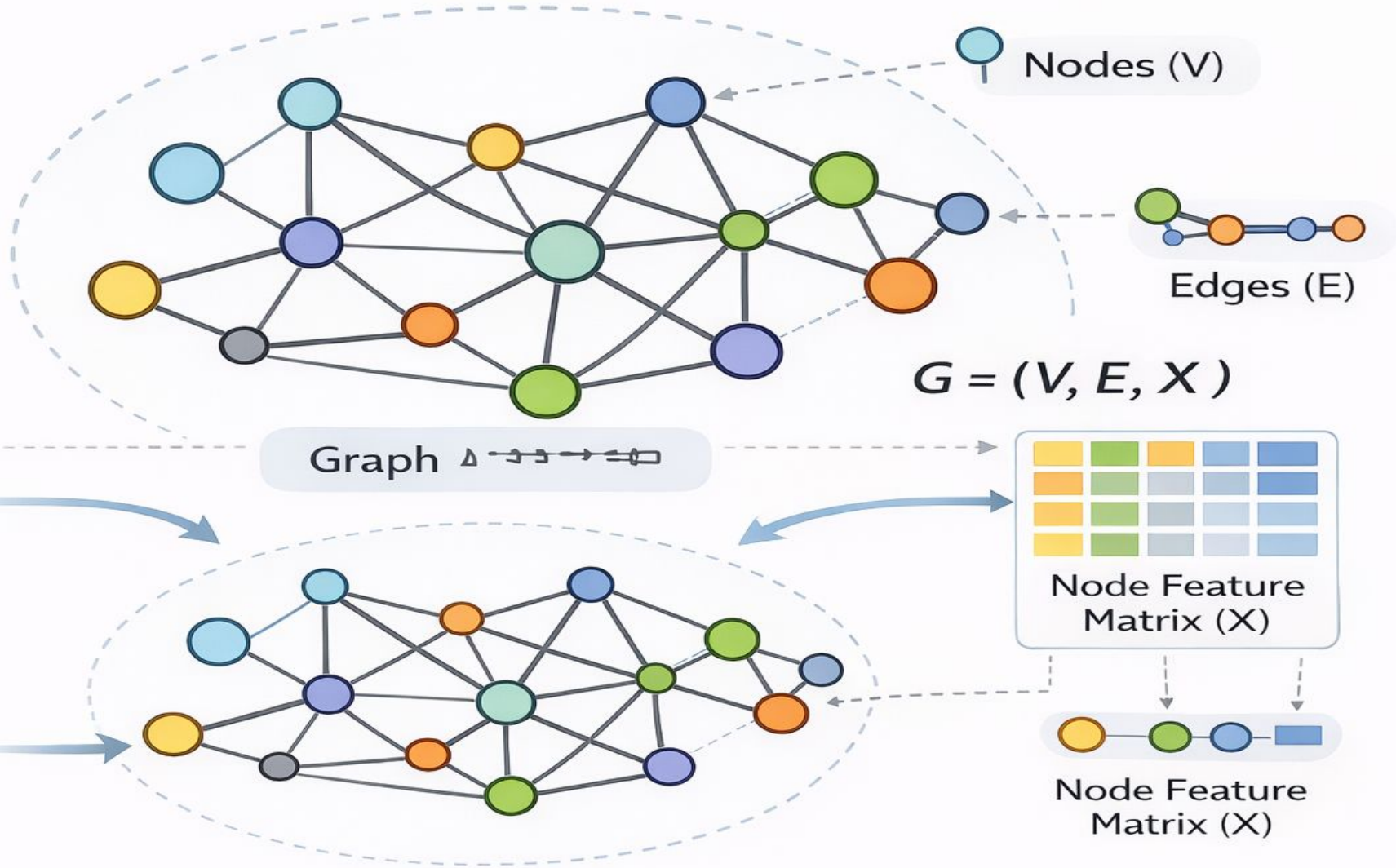
$$G = (V, E, X)$$

- $V \rightarrow$ Nodes (entities)
- $E \rightarrow$ Edges (relationships)
- $X \rightarrow$ Node feature matrix

Social Network



Traffic Network



② Neighborhood Aggregation (Message Passing)

Each node gathers information from its neighbors.

General formulation:

$$h_v^{(k)} = \sigma \left(W^{(k)} \cdot \{h_u^{(k-1)}, u \in \mathcal{N}(v)\} \right)$$

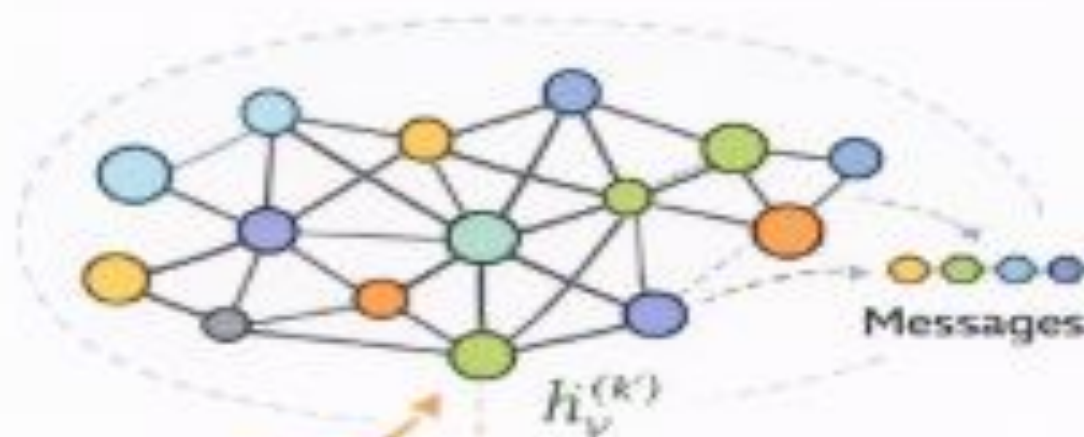
Where:

- $h_v^{(k)} \rightarrow$ Node representation at layer k
- $\mathcal{N}(v) \rightarrow$ Neighbors of node v
- AGG $\rightarrow$ Aggregation function (mean, sum, max)
- $W^{(k)} \rightarrow$ Learnable weight matrix
- $\sigma \rightarrow$ Activation function

This process allows each node to learn from its local structure.

② Neighborhood Aggregation (Message Passing)

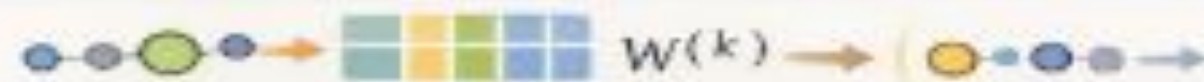
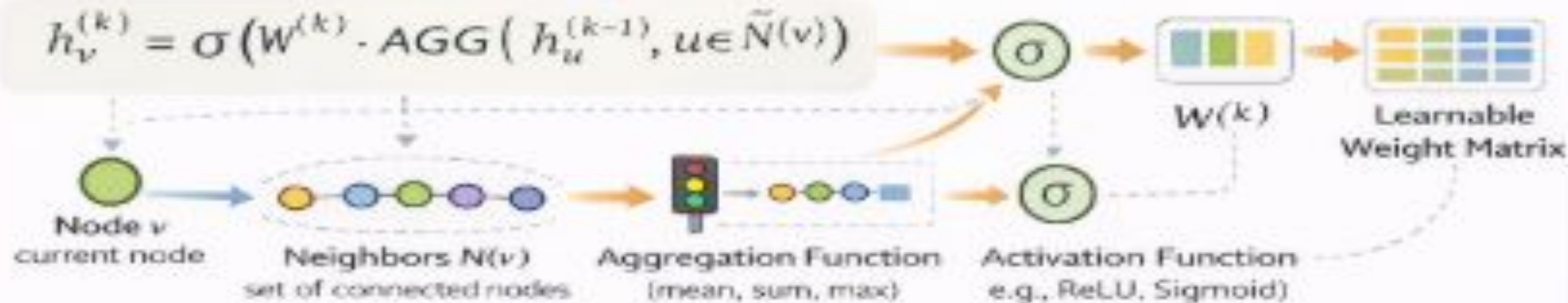
Each node gathers information from its **neighbors**.



$h_v^{(k)}$: Node representation at layer k
 $N(v)$: Neighbors of node v
AGG: Aggregation function (mean, sum, max)
 $W^{(k)}$: Learnable weight matrix
 σ : Activation function

General Formulation

$$h_v^{(k)} = \sigma(W^{(k)} \cdot \text{AGG}(h_u^{(k-1)}, u \in N(v)))$$



③ Representation Learning (Embedding Generation)

After multiple layers, nodes are transformed into dense vector representations:

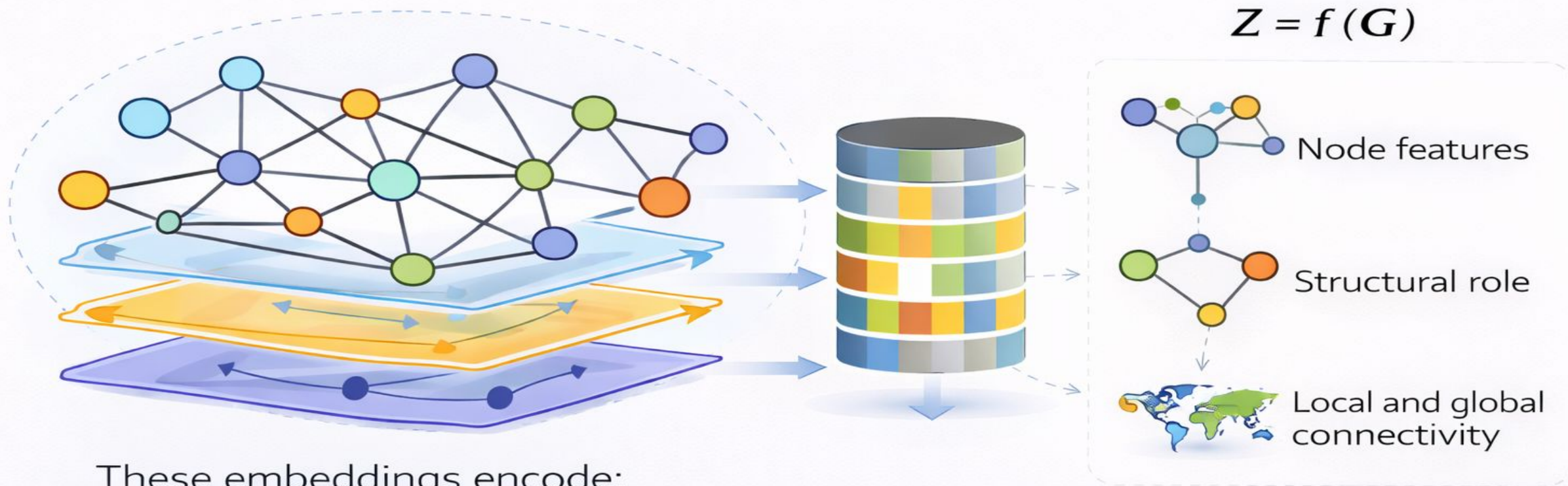
$$Z = f(G)$$

These embeddings encode:

- Node features
- Structural role
- Local and global connectivity

③ Representation Learning (Embedding Generation)

After multiple layers, nodes are transformed into dense vector representations: $Z = f(G)$



These embeddings encode:

- Node features
- Structural role
- Local and global connectivity

④ Task-Specific Prediction

Task	Function
Node Classification	Predict node labels
Link Prediction	Predict missing edges
Graph Classification	Predict label for entire graph
Community Detection	Identify clusters

⑤ Optimization (Training Phase)

- ❑ Loss function example (Node classification):

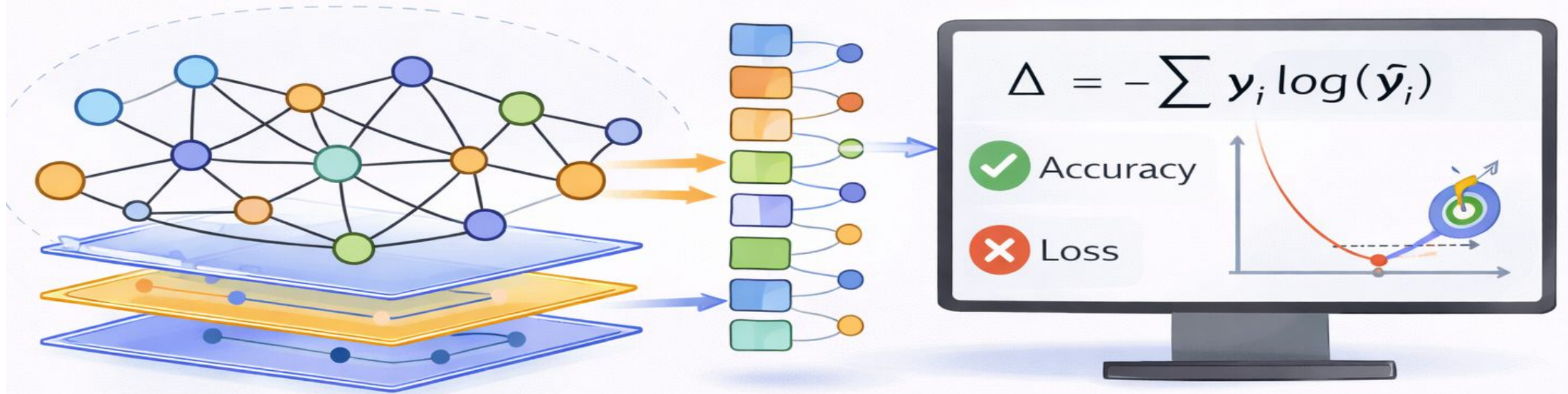
$$\mathcal{L} = - \sum_{i \in \mathcal{Y}_L} y_i \log(\hat{y}_i)$$

- ❑ The model learns by minimizing prediction error while preserving graph structure.

⑤ Optimization (Training Phase)

Loss function example (Node classification):

$$\Delta = \sum y_i \log(\hat{y}_i)$$



The model learns by minimizing prediction error while preserving graph structure.

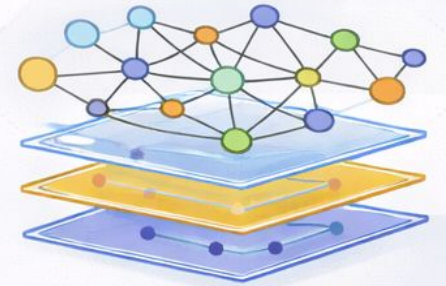
End-to-End Functional Flow



Graph Data (Nodes + Edges + Features)



Neighborhood Aggregation



Layered Graph Transformations



Node/Graph Embeddings



Task-Specific Output



Node Classification



Link Prediction



Graph Classification



Community
Detection

Graph Statistics and Kernel Methods

- ❑ Graph statistics and kernel methods are important techniques used in machine learning and data analysis for handling **graph-structured data**.
- ❑ A graph is a mathematical structure made up of **nodes (vertices)** and **edges (links)**, which is used to represent relationships among objects.
- ❑ Examples include social networks, biological networks, communication systems, transportation networks, and molecular structures.

simple mathematical expression for that statement is:

$$G = (V, E)$$

where:

- G = Graph
- V = set of **vertices (nodes)**
- E = set of **edges (links)** connecting pairs of vertices

For example,

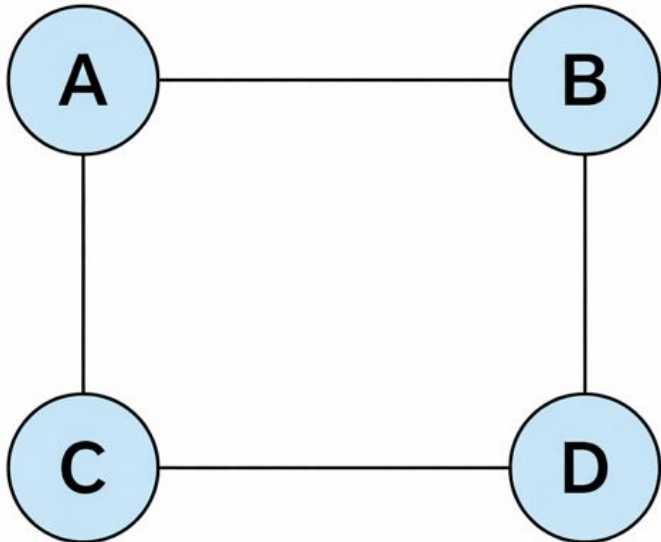
$$V = \{A, B, C, D\}$$

$$E = \{(A, B), (A, C), (B, D), (C, D)\}$$

So the graph can be written as:

$$G = (V, E) = (\{A, B, C, D\}, \{(A, B), (A, C), (B, D), (C, D)\})$$

➤ Graph consists of nodes and the links between them.



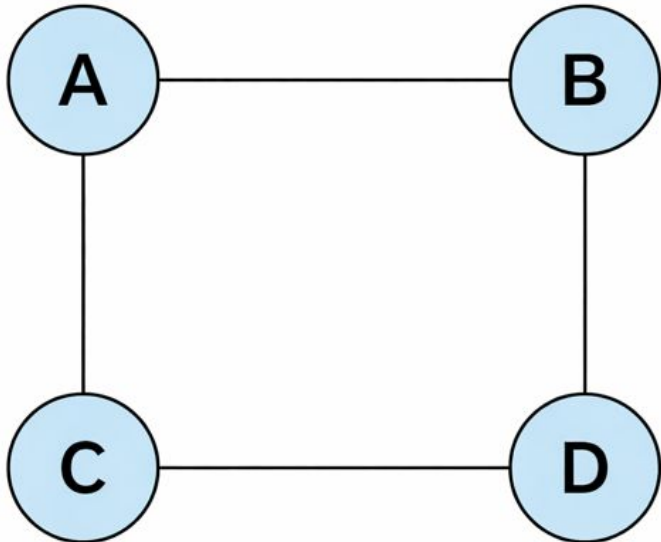
Density of the graph tells how many edges exist compared to the maximum possible edges.

- Number of nodes = 4
- Maximum possible edges = $4 \times 3 / 2 = 6$
- Actual edges = 4
- Density = $4 / 6 = 0.67$

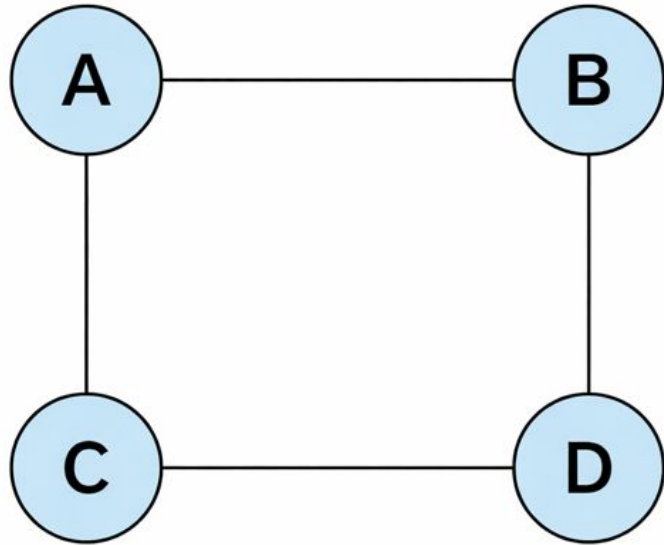
Shortest path

- From A to D, the shortest path can be:
 - $A \rightarrow B \rightarrow D$
 - or $A \rightarrow C \rightarrow D$
- So the shortest path length is 2.

- ❑ **Kernel methods** are machine learning techniques that measure similarity between data objects. In graph-based problems, kernel methods are used to compare graphs, nodes, or substructures without converting them directly into simple vectors.
- ❑ A **graph kernel** computes similarity by examining structural patterns such as walks, paths, subtrees, or subgraphs.
- ❑ This is especially useful in classification tasks like molecule classification, protein function prediction, and social network analysis.



Graph 1:



Graph 2:



- ❑ Graph 1 has 4 nodes and 4 edges, while Graph 2 has only 2 nodes and 1 edge.
- ❑ A **graph kernel** compares these two graphs by checking structural similarity.
- ❑ For example:
 - ❑ Graph 1 has more connections
 - ❑ Graph 2 is much simpler
 - ❑ So their similarity score will be **low**
 - ❑ But if we compare Graph 1 with another square-shaped graph, the similarity score will be **high**

- **Graph statistics** tell us the properties of one graph
- **Kernel methods** tell us how similar two graphs are

example

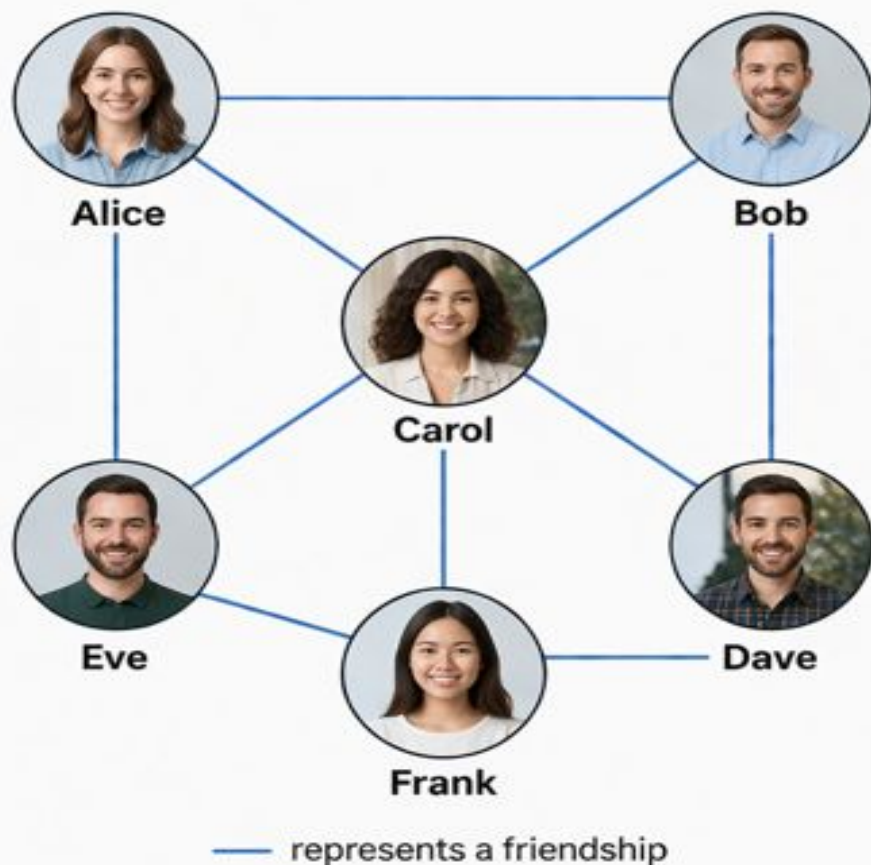
- In a social network, graph statistics identify important users and connectivity patterns, while kernel methods compare two different social groups based on their structure.

Social Network Analysis: Real-Life Problem and Solutions

A social media company wants to analyze a small part of its user network to understand the structure of friendships and identify important people who can help spread information quickly.

Problem: Understand the Social Network

- **Who** are the most connected users?
- **Who** acts as a **bridge** between different people?
- **How** tightly connected is this group?



Solutions with Graph Analysis

1. Find the Most Connected Users (Degree)

Count the number of friends (connections) for each user.

User	Friends	Degree
Alice	Bob, Carol, Eve	3
Bob	Alice, Carol, Dave	3
Carol	Alice, Bob, Dave, Eve, Frank	5
Dave	Bob, Carol, Frank	3
Frank	Carol, Dave, Eve	3

✓ **Result:** Carol is the most connected user.

2. Find the Bridge/Connector (Betweenness Centrality)

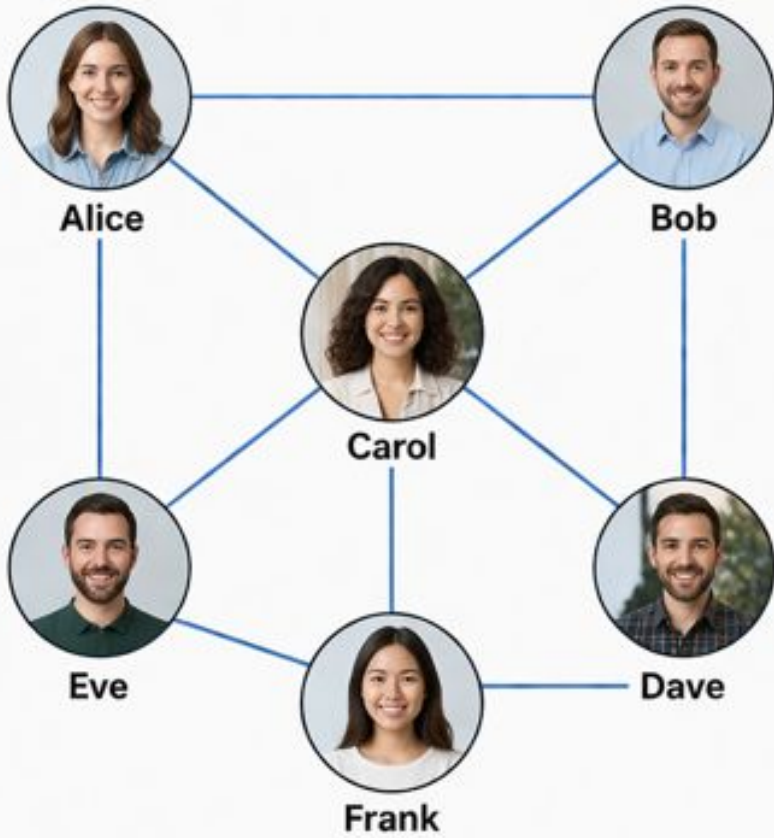
Identify who lies on the shortest paths between others.

✓ **Result:** Carol connects Alice–Bob, Alice–Dave, Bob–Eve,
→ Carol acts as the **main bridge** of the network.

3. Check How Tight the Group Is (Density & Clustering)

- Density measures how many connections exist out of all possible connections.
- Clustering shows how close friends of a person are to each other.

✓ **Result:** The network has **moderate** density with a clear cluster (Alice–Carol–Eve are well connected).



Number of nodes $n = 6$

$\{Alice, Bob, Carol, Dave, Eve, Frank\}$

Number of edges $e = 10$

Visible edges are:

1. Alice — Bob
2. Alice — Carol
3. Alice — Eve
4. Bob — Carol
5. Bob — Dave
6. Carol — Dave
7. Carol — Eve
8. Carol — Frank
9. Eve — Frank
10. Frank — Dave

$$\text{Density} = \frac{2e}{n(n-1)}$$

Substitute the values:

$$\text{Density} = \frac{2 \times 10}{6(6-1)} = \frac{20}{30} = \frac{2}{3}$$

$$\text{Density} \approx 0.667$$

Interpretation

This means the graph contains about **66.7% of all possible connections** among the nodes.

So, the network is **moderately high to strongly connected**.

- ❑ If density is close to **1**, almost every node is connected to every other node.
- ❑ If density is close to **0**, the graph is sparse, with very few connections.
- ❑ Here, **0.667** shows that the people in the network are fairly well connected, but not fully connected.

General graph kernel form

$$K(G_1, G_2) = \phi(G_1) \cdot \phi(G_2)$$

where:

- ❑ G_1 and G_2 are two graphs
- ❑ $\phi(G)$ is the graph feature representation

Example using your social network graph

Suppose:

$$\phi(G) = [\textit{number of nodes} \ \textit{number of edges} \ \textit{density}]$$

For your graph:

- nodes = 6
- edges = 10
- density = 0.667

So,

$$\phi(G_1) = [6, 10, 0.667]$$

Suppose another graph has:

- nodes = 6
- edges = 9
- density = 0.6

Then,

$$\phi(G_2) = [6, 9, 0.6]$$

Using a linear kernel:

$$K(G_1, G_2) = \phi(G_1) \cdot \phi(G_2)$$

$$K(G_1, G_2) = (6)(6) + (10)(9) + (0.667)(0.6)$$

$$K(G_1, G_2) = 36 + 90 + 0.4002$$

$$K(G_1, G_2) \approx 126.4$$

Interpretation

- ❑ The value $K(G_1, G_2) \approx 126.4$ represents the similarity between two graphs, and a high value indicates that the two graphs have closely related structural features such as nodes, edges, and density.

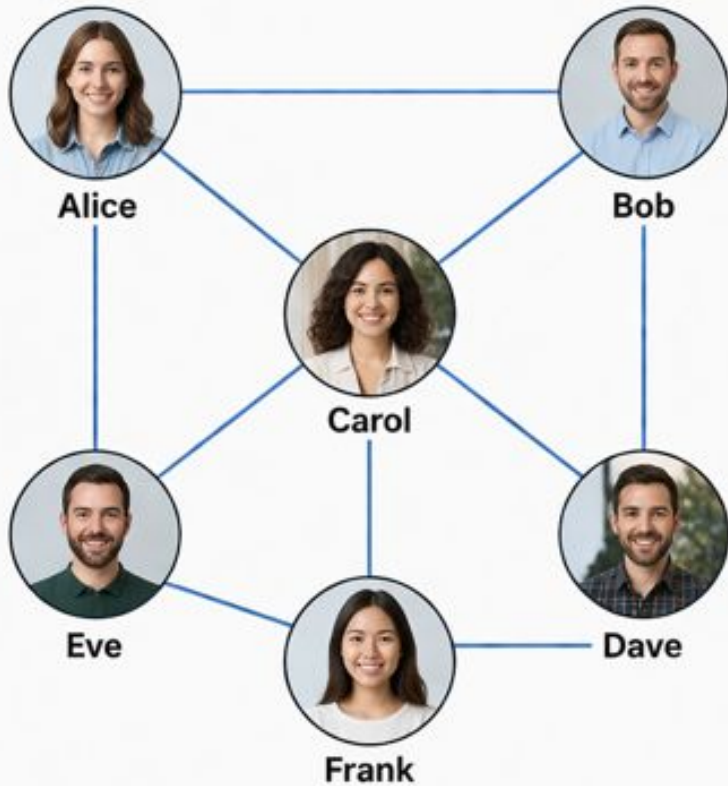
Neighborhood Overlap Detection

❑ **Neighborhood Overlap Detection** is a graph analysis method used to measure how strongly two nodes are related based on their **common neighbors**.

$$NO(u, v) = \frac{|N(u) \cap N(v)|}{|N(u) \cup N(v)|}$$

where:

- $N(u)$ = neighbors of node u
- $N(v)$ = neighbors of node v
- $N(u) \cap N(v)$ = common neighbors
- $N(u) \cup N(v)$ = total unique neighbors



Example

Suppose:

$$N(\text{Alice}) = \{\text{Bob}, \text{Carol}, \text{Eve}\}$$

$$N(\text{Bob}) = \{\text{Alice}, \text{Carol}, \text{Dave}\}$$

Common neighbors:

$$N(\text{Alice}) \cap N(\text{Bob}) = \{\text{Carol}\}$$

Union:

$$N(\text{Alice}) \cup N(\text{Bob}) = \{\text{Alice}, \text{Bob}, \text{Carol}, \text{Dave}, \text{Eve}\}$$

So,

$$NO(\text{Alice}, \text{Bob}) = \frac{1}{5} = 0.2$$

This means Alice and Bob have a small neighborhood overlap.

Real-life interpretation

If two people share many mutual friends, their neighborhood overlap is high. This means they are likely part of the same social circle.

Graph Laplacians and Spectral Methods

- ❑ **Graph Laplacians and Spectral Methods** are important concepts in graph theory and machine learning used to study the structure of a graph through matrices and eigenvalues.

Graph Laplacian

For a graph $G = (V, E)$, the Graph Laplacian matrix is defined as:

$$L = D - A$$

where:

- . D = Degree matrix
- . A = Adjacency matrix

Degree matrix

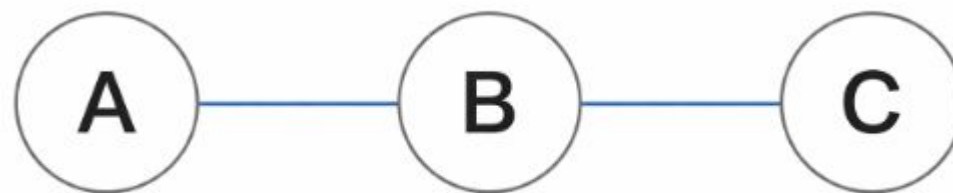
D is a diagonal matrix containing node degrees.

Adjacency matrix

A shows whether two nodes are connected or not.

Example

Suppose the graph has 3 nodes:



Adjacency matrix: $A = \begin{bmatrix} 0 & 1 & 0 \\ 1 & 0 & 1 \\ 0 & 1 & 0 \end{bmatrix}$

Degree matrix: $D = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 2 & 0 \\ 0 & 0 & 1 \end{bmatrix}$

So Laplacian matrix: $L = D - A = \begin{bmatrix} 1 & -1 & 0 \\ -1 & 2 & -1 \\ 0 & -1 & 1 \end{bmatrix}$

Interpretation

- ❑ This matrix represents the **Graph Laplacian** of the simple graph $A - B - C$.
- ❑ It is obtained by subtracting the adjacency matrix A from the degree matrix D , that is, $L = D - A$.
- ❑ In this matrix, the **diagonal values** (1, 2, 1) show the degree of each node, meaning node A has 1 connection, node B has 2 connections, and node C has 1 connection.
- ❑ The **off-diagonal values** (-1) indicate that two nodes are directly connected, such as A with B and B with C , while **0** indicates no direct connection, such as between A and C .
- ❑ Therefore, this Laplacian matrix clearly describes both the connectivity and the structure of the graph, and it is very useful in graph analysis, clustering, and spectral methods in machine learning.

Spectral Methods

Spectral Methods are graph analysis techniques that study a graph using the eigenvalues and eigenvectors of a matrix such as the adjacency matrix or, most commonly, the graph Laplacian matrix.

For a graph $G = (V, E)$, the Laplacian matrix is

$$L = D - A$$

where:

- D = degree matrix
- A = adjacency matrix

Then spectral methods analyze the equation

$$Lx = \lambda x$$

where:

- λ = eigenvalue
- x = eigenvector

Spectral methods are used to study:

- ☐ connectivity of the graph
- ☐ clustering or community structure
- ☐ graph partitioning
- ☐ node similarity
- ☐ smooth flow of information across the graph

Multi-Relational Data and Knowledge Graphs

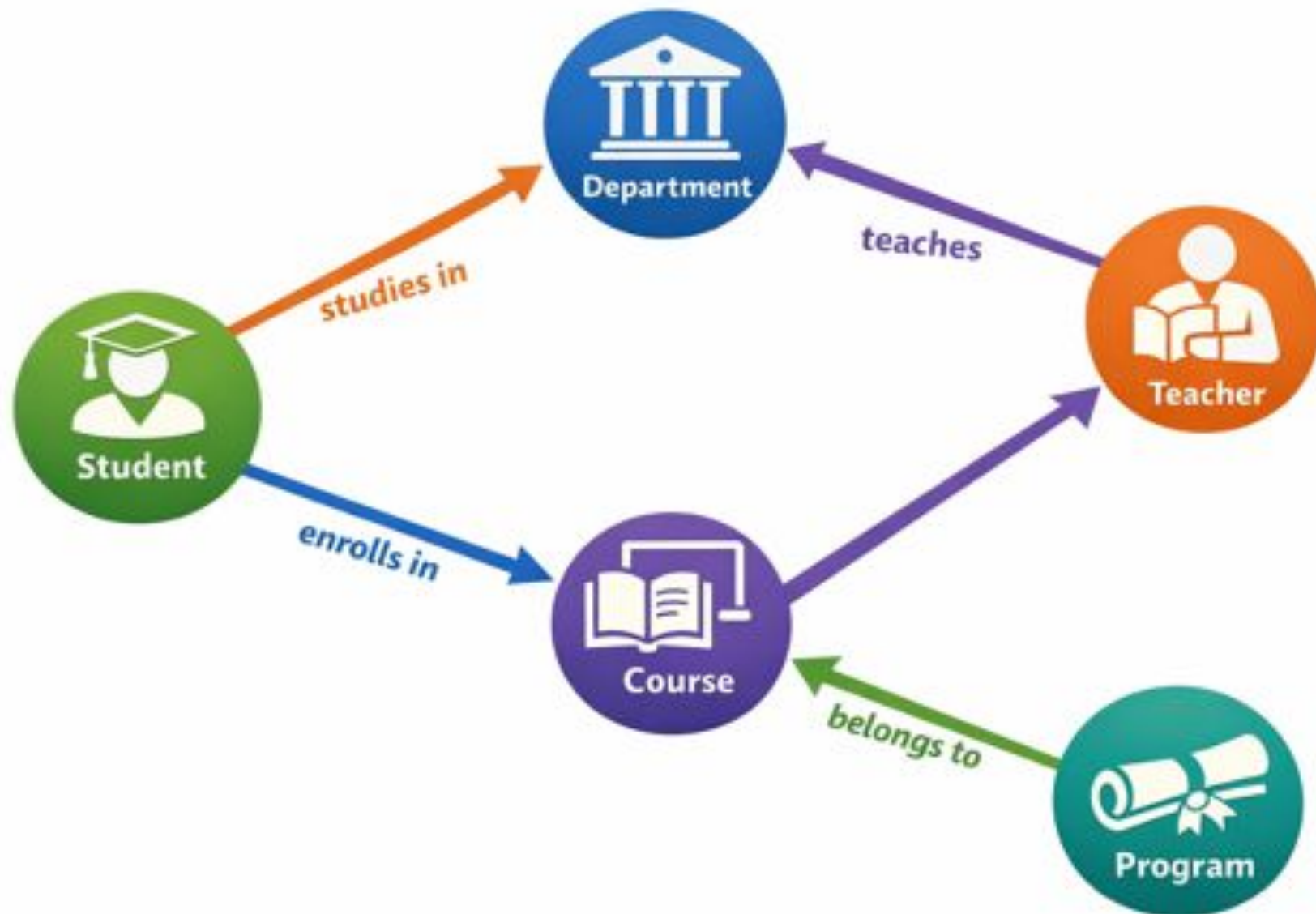
❑ **Multi-relational data** is data in which objects are connected through **different types of relationships**, not just one single relation. In ordinary data tables, information is often stored in rows and columns. But in real-world systems, entities are linked in many ways at the same time.

❑ For example, in a university domain:

- ❑ a **student** studies in a **department**
- ❑ a **teacher** teaches a **course**
- ❑ a **student** enrolls in a **course**
- ❑ a **course** belongs to a **program**

Here, the same set of entities can be connected by many different relation types. This is called **multi-relational data**.

Multi-Relational Data in University Domain



knowledge graph

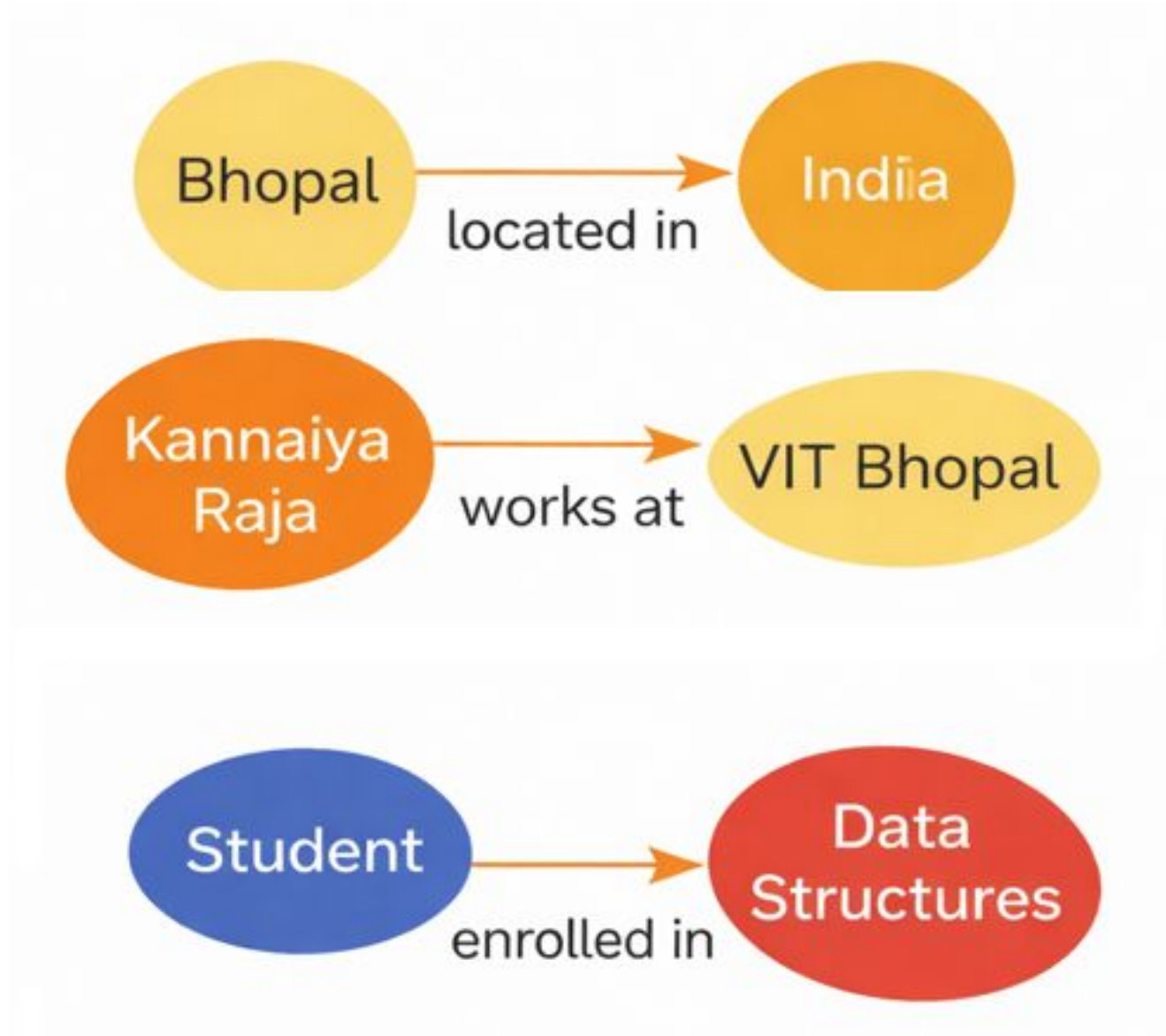
- ❑ A knowledge graph is a structured representation of knowledge in which **entities** are represented as nodes and **relationships** are represented as edges between those nodes.

For example:

- ❑ Bhopal — located in — India
- ❑ Raja — works at — VIT Bhopal
- ❑ Student — enrolled in — Data Structures

A knowledge graph helps organize information in a connected and meaningful form.

knowledge graph



Connection between multi-relational data and knowledge graphs

- ❑ A knowledge graph is one of the most common ways to represent **multi-relational data**.

In other words:

- ❑ **multi-relational data** is the type of data
 - ❑ **knowledge graph** is a graph-based way to model that data
- ❑ So Knowledge graphs are a natural representation for data containing multiple relation types.